

Recursividade, Teste de Mesa e Ferramentas de Algoritmos

11. Recursividade

A recursividade é uma técnica de programação na qual uma função chama a si mesma para resolver um problema. Para evitar execução infinita, é necessário estabelecer uma **condição de parada** (caso base). Um exemplo clássico é o cálculo do fatorial:

```
function fatorial(n) {
  if (n === 0) return 1;
  return n * fatorial(n - 1);
}
```

A função se chama até que **n === 0**, quando a condição de parada é atingida e os resultados começam a ser retornados. A recursividade é útil em problemas divisíveis em subproblemas semelhantes, como percorrer árvores, realizar buscas e resolver problemas matemáticos. Entretanto, seu uso inadequado pode consumir mais memória e processamento.

12. Teste de Mesa

O teste de mesa é uma técnica para verificar manualmente o funcionamento de um algoritmo, acompanhando passo a passo a execução das instruções e os valores das variáveis. Normalmente, são registrados em uma tabela os valores das variáveis durante cada etapa do algoritmo. Dessa forma, é possível identificar erros na lógica antes de implementar ou executar o programa.

Para um algoritmo que soma dois números:

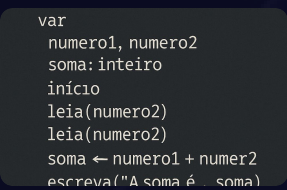
```
let A = 5;
let B = 3;
let Soma = A + B;
```

Etapa	A	B	Soma
Inicial	5	3	—
Soma	5	3	8

Essa técnica é especialmente útil para estudantes e desenvolvedores, pois facilita a compreensão da lógica e ajuda a encontrar erros de processamento.

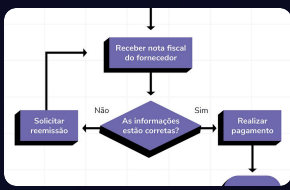
13. Ferramentas para Elaboração de Algoritmos

As ferramentas para elaboração de algoritmos auxiliam na representação, organização e visualização da lógica de uma solução antes de sua implementação em uma linguagem de programação. Entre as principais ferramentas estão:



Pseudocódigo

Utiliza uma linguagem estruturada e próxima da linguagem humana para representar as etapas de um algoritmo, facilitando o planejamento da solução.



Fluxograma

Representa graficamente as etapas do algoritmo por meio de símbolos e setas. O fluxograma é, na verdade, um **tipo específico** de diagrama de fluxo focado na sequência lógica e operacional de tarefas

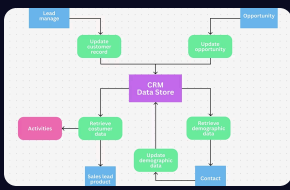
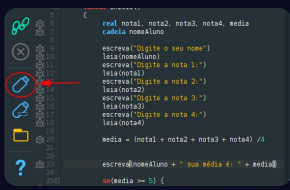


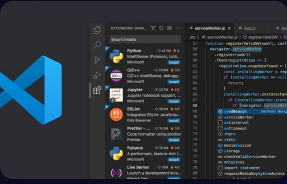
Diagrama de Fluxo

Permite representar a sequência de operações, decisões e possíveis caminhos de um processo, organizando visualmente as etapas do algoritmo.



Portugol

Utiliza uma estrutura semelhante às linguagens de programação, mas com comandos em português, sendo bastante utilizado no ensino de lógica de programação.



IDE

Ambiente de Desenvolvimento Integrado que permite escrever, executar, testar e corrigir programas, sendo útil principalmente na etapa de implementação do algoritmo.

Essas ferramentas ajudam a planejar a solução, identificar erros de lógica e facilitar a compreensão do funcionamento do algoritmo antes ou durante o desenvolvimento do programa.

Diferenças entre Fluxograma e Diagrama de Fluxo

Fluxograma

O fluxograma utiliza **símbolos padronizados** com significados específicos: elipse para início/fim, retângulo para processos, losango para decisões e setas para o fluxo de controle. É ideal para representar a lógica geral de um algoritmo de forma visual e acessível.

```
// Exemplo em JavaScript
let x = 10;
if (x > 5) {
  console.log("Maior");
} else {
  console.log("Menor");
}
```

Diagrama de Fluxo

O diagrama de fluxo é uma representação mais **detalhada e técnica**, frequentemente utilizado em engenharia de software e processos empresariais. Enquanto o fluxograma foca na lógica do algoritmo, o diagrama de fluxo pode incluir paralelismo, subprocessos e múltiplos caminhos de forma mais explícita.

```
// Exemplo com múltiplos caminhos
let nota = 75;
if (nota >= 90) {
  console.log("A");
} else if (nota >= 70) {
  console.log("B");
} else {
  console.log("C");
}
```

Dica: Use o teste de mesa para validar algoritmos recursivos — registre o valor de *n* em cada chamada e acompanhe o retorno do caso base até o resultado final.